

Automatisation d'un Revox B77



Télécommande par page WEB /
ESP8266
utilisation de Websockets

Le fameux Revox A77

- Willi Studer crée en 1948 Dynavox qui devient plus tard REVOX fabriquant des enregistreurs à bande.
- Studer 27 est le premier enregistreur à être utilisé par la radio Suisse (1952)
- Suivent le B36, premier enregistreur à 3 têtes!
- En 1966, Revox déménage en Allemagne mais construit tout de même une usine à Regensdorf (CH) en 1967.
- En 1967 le G36 fut mis sur le marché, premier enregistreur à transistors, suivi du A77 enregistreur à entraînement direct par Cabestan, moteur servo-régulé permettant un déroulement très régulier de la bande m.
- Plus de 80'000 pièces Série 36 furent vendues (source wikipedia).

Rem: Le A77 fut le premier à être complété par un Amplificateur A78 et un Tuner A76

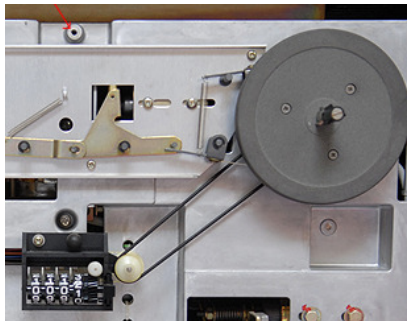
Kudelski fut fondée en 1951 et créa le premier enregistreur portable Nagra I

Suivit le B77

- En 1970, le A700 fut le produit top, machine 3 vitesses dont le moteur est régulé par PLL quartz. Suit le B77, modèle à commandes électroniques et non mécaniques (A77)



B77 (1979)



compteur à couroi
(compte tours)



PR99

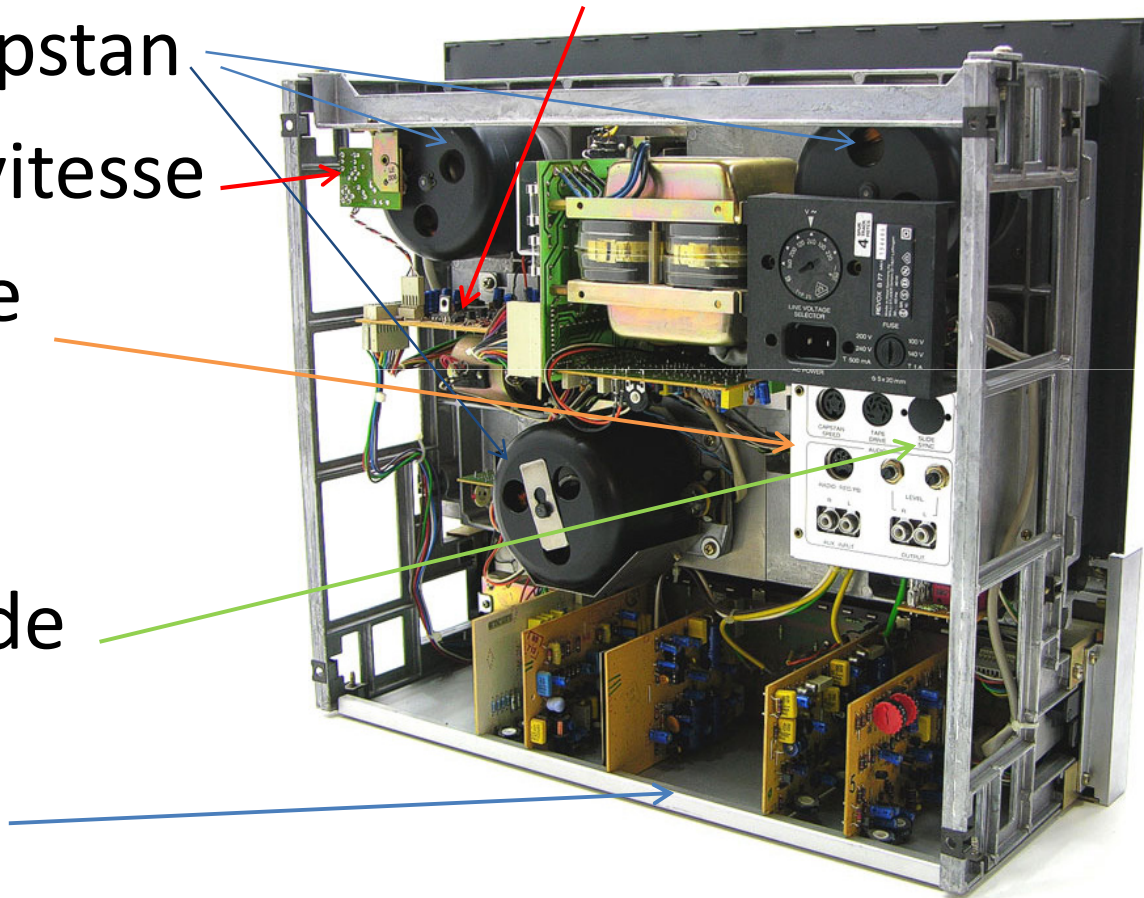


Version semi-pro avec
entrée/sorties analogiques
balancées et compteur
temps réel digital

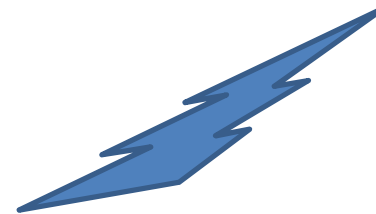
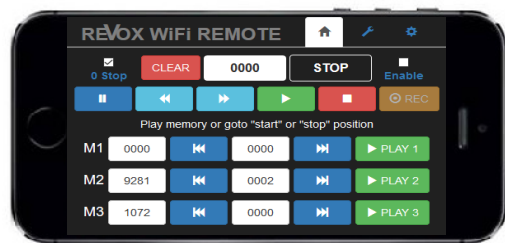
B77 Fonctionnement

1. 3 moteurs capstan
2. Contrôle de vitesse
3. Entrées sortie Cinch/DIN
4. Prise télécommande
5. Electronique modulaire

Contrôleur entrainement



Mon projet



Télécommander les fonctions
de déplacement du B77

Les fonctions audio (volume, entrées)
ne sont pas télécommandables sans
modifications importantes

Ma télécommande

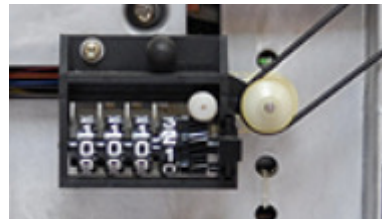
Cahier de charges



1. Télécommander l'enregistreur par interface WEB sans modifier l'enregistreur (ESP8266)



2. Compteur 4 digits remplacent le compteur à courroi sur panneau avant. Mémorisation de la position à l'arrêt de l'enregistreur !!

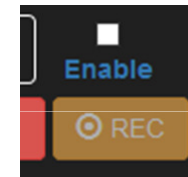


Fonctionnalité



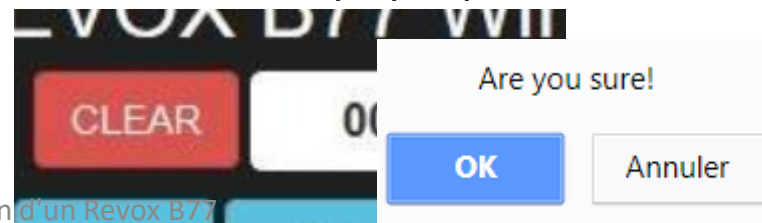
- Protection de la touche « enregistrer/record)

En pratique il faut presser 2 touches sur l'enregistreur pour démarrer un enregistrement ce qui n'est pas possible avec une page WEB.



- Protection de la touche « reset compteur»

Sur l'enregistreur il faut presser sur un bouton à côté du compteur, sur la page web, le bouton est à côté des autres boutons, il y a donc danger de mettre le compteur à zéro par erreur. Une fenêtre pop-up demandera donc une confirmation.



Fonctionnalité

Suite



- Mémorisation de positions de morceaux enregistrés (début....fin, aller au début, reproduit la plage)



- Editeur de plages d'enregistrement sur une page séparée pour éviter de presser sur une fausse touche lors de la reproduction



Problèmes

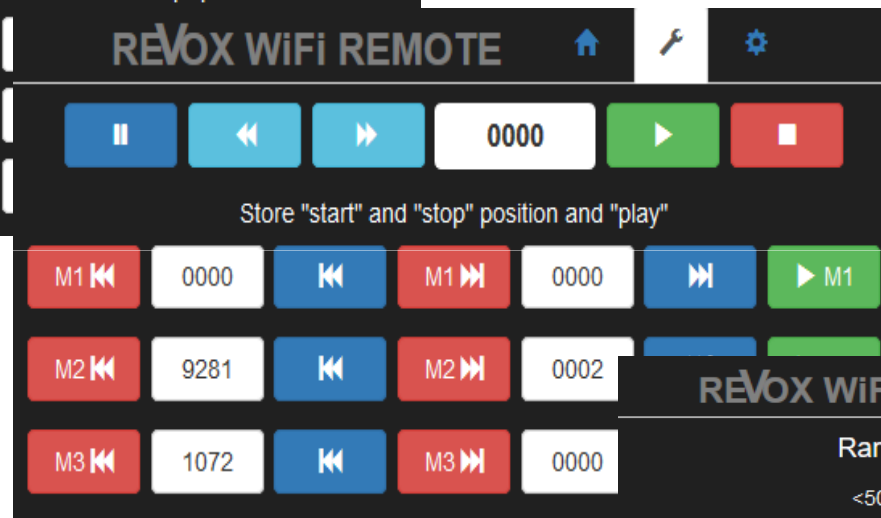


- Positionnement précis de l'arrêt sur position
 - Notre enregistreur ne permet pas un arrêt précis pour les fonctions forward/rewind/stop, ces commandes prennent plusieurs tours jusqu'à l'arrêt et la distance dépend de la distance parcourue !
- Solution:
 - Ajouter ou soustraire la valeur d'une table, valeur proportionnelle à la distance parcourue à la distance à parcourir. Exemple: si la distance d'arrêt sur 500 tours en avant est de 10 tours, nous déduisons 10 tours à la position à parcourir = stop à 490 tours de compteur.
 - Cette information sera enregistrée en EEPROM, donc disponible après arrêt et remise en marche de l'appareil.

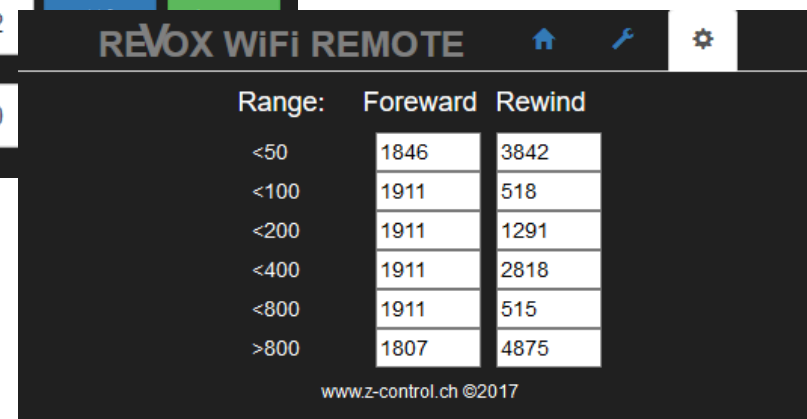
Design de l'interface WEB



Page principale, commandes
(positions non-modifiables)



Page Enregistrement des
positions



Page calibration de l'arrêt de bande

Communication

Client WEB ----- B77



1. Demande de connexion depuis le client -----
2. Envoi de la page web depuis l'enregistreur (ESP8266) -----
3. Le client affiche la page selon le code HTML + CSS
4. Le client (mobile) démarre le Javascript
5. Le code de l'ESP fait la mise à jour du compteur (au changement)
6. Le code Javascript envoie les informations concernant les touches activées sur le portable

Schéma entrainement

Revox B77/MKII

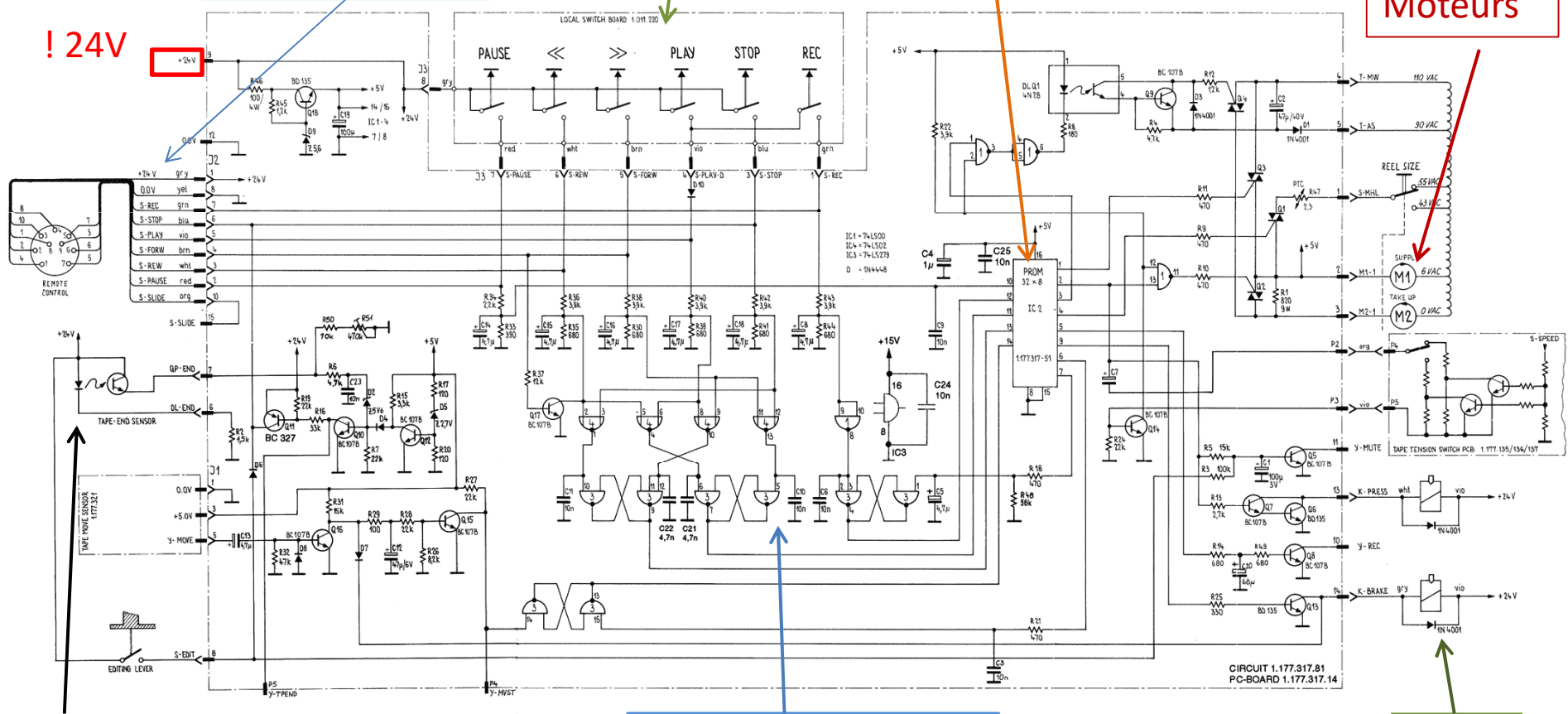
VERSION 1.177.317.00
 C23 = NOT EQUIPPED
 D02 = Z15V
 P05 = NOT EQUIPPED
 R02 = 3,3k
 R50 = NOT EQUIPPED
 R51 = NOT EQUIPPED
 PCB = 1.177.317.12

Interface
télécommande

Boutons frontaux

EEPROM = table d'état

Puissance
Moteurs



! 24V

Détection fin de bande

Logique des boutons

Freins

Télécommande Revox B77

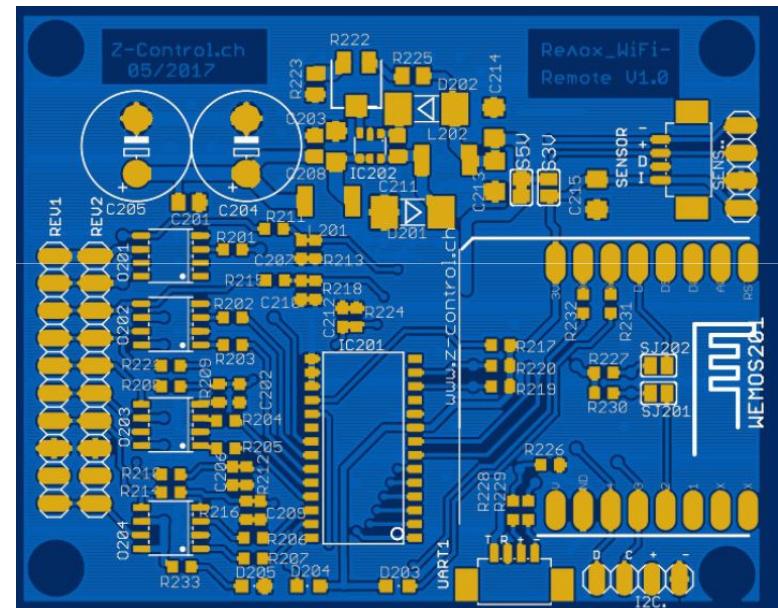
Circuit principal

1. Carte enfichable sur le connecteur interne de la télécommande (opto-coupleurs pour tirer les lignes de 24V sur GND).

- a) ESP8266 sur socle version Wemos avec interface USB, plus facile pour programmer.
- a) Interface MCP23017 pour augmenter le nombre de lignes digitales sur ESP8266 (8 lignes en entrée et 8 lignes en sortie). Configuration de la fonction interruption pour gérer les changements à l'entrée (pression de touches de la plaque frontale).

Interfaçage des sorties sur 24V par opto-coupleurs.

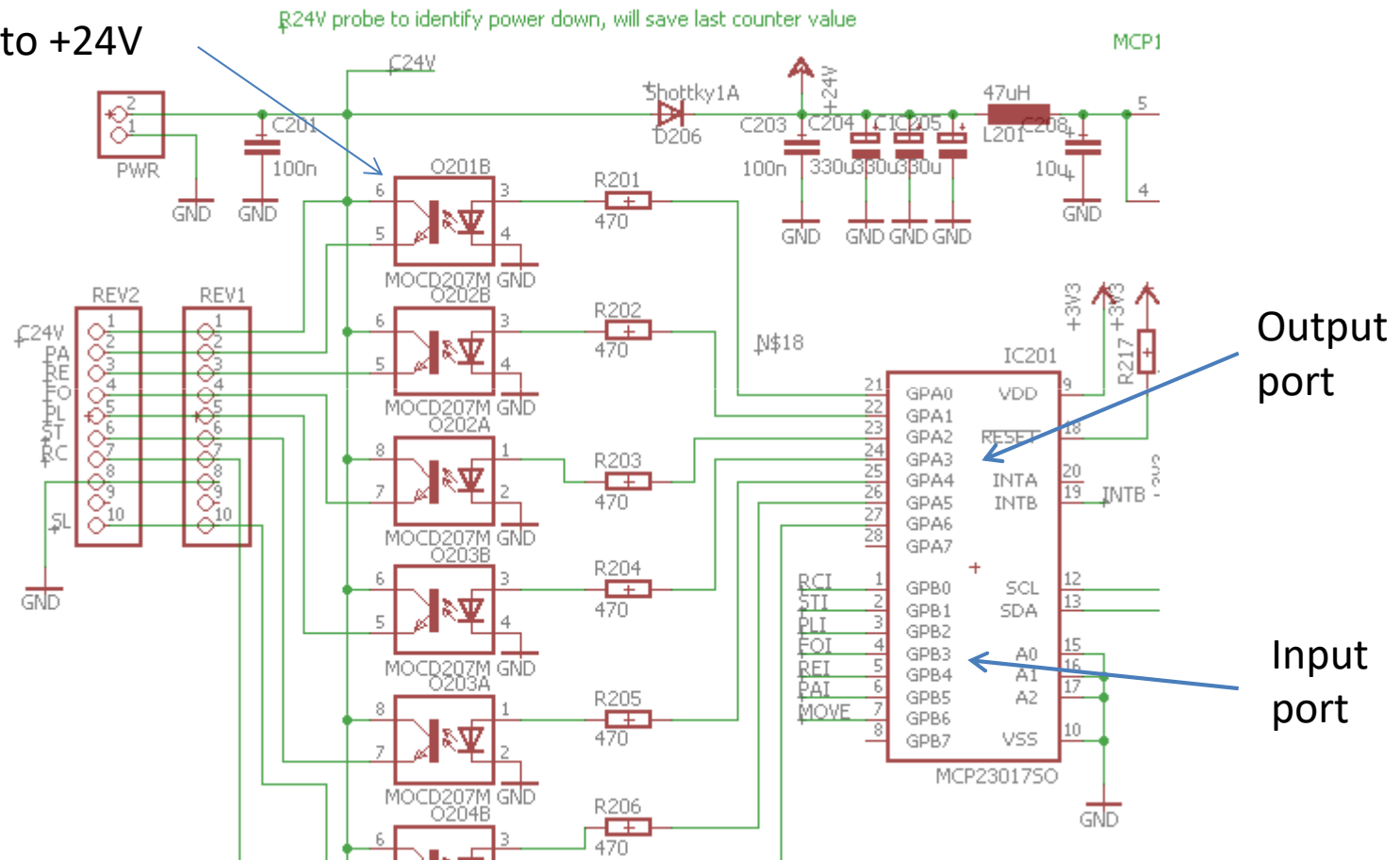
- a) Alimentation à découpage pour réduit la tension de 24V à 5V
- b) Détection de coupure de courant pour mémoriser la position



Télécommande Revox B77

Contrôleur

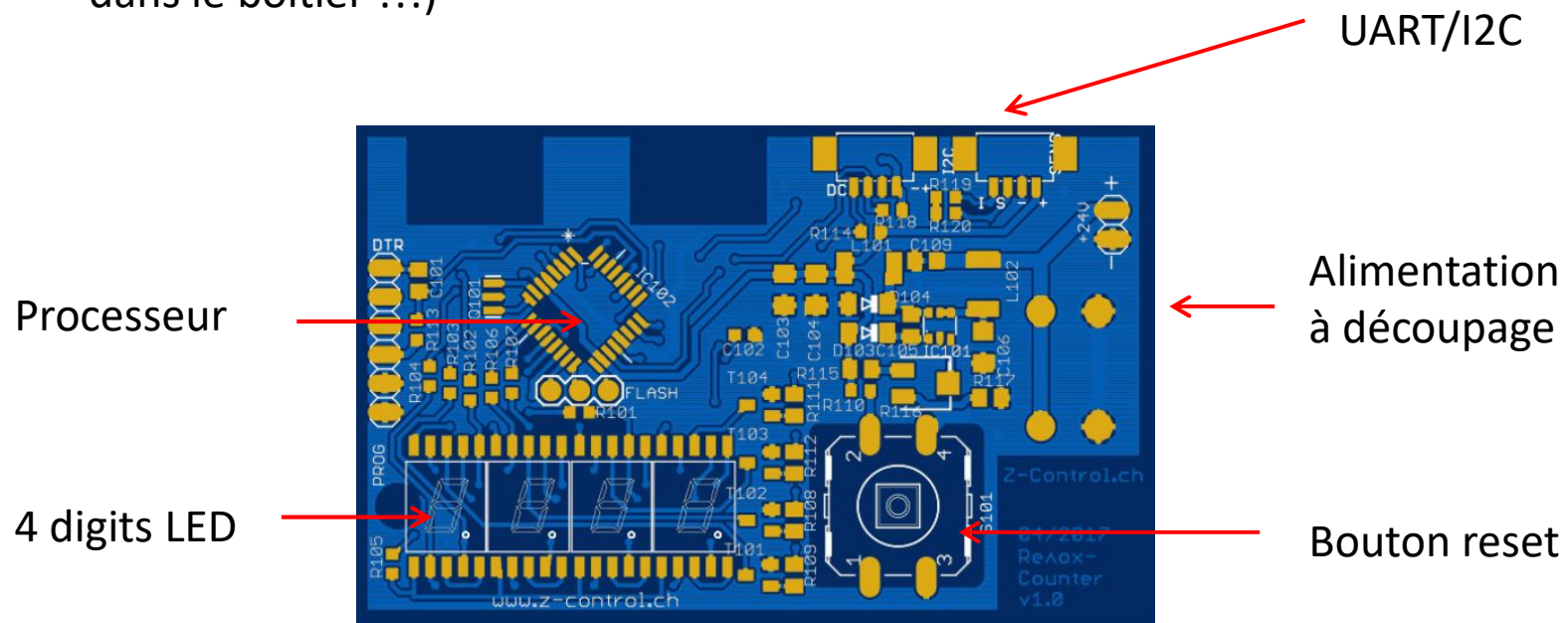
Opto=pull-up to +24V



Télécommande Revox B77

Hardware

1. Carte pour compteur sur plaque frontale (4 digits)
 - a) Bouton de reset
 - b) Atmega 328 -> utilisation de librairie existantes pour l'affichage
 - c) Interface série vers la carte (1), i2c en option, io capteur de hall possible
 - d) Ajustement sur la fenêtre existante de l'enregistreur (pas de modification dans le boitier !!!)



Télécommande Revox B77

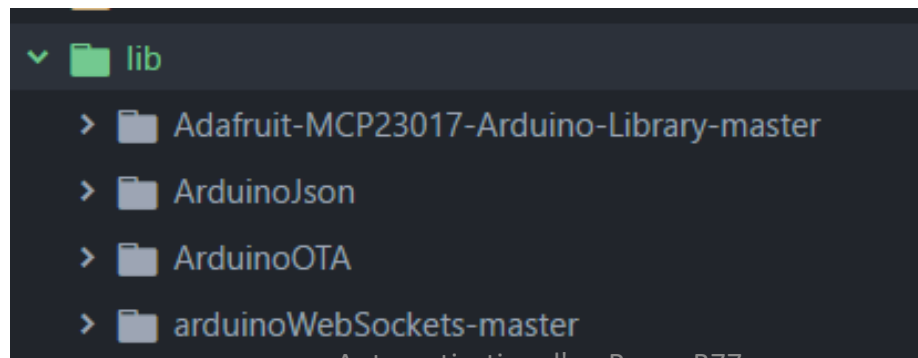
Software: Routines/Fonctions

- Setup (dans ino):
 - Init WEBServer, OTA, SPIFFS, Wifi, MDNS, EEPROM
 - Restauration des compteurs, offsets
 - Affichage du compteur sur circuit
 - Initialisation de l'interruption pour capteurs de hall
- Loop (dans ino)
 - Traitement WEBsockets, OTA, WIFI (pooling)
 - Traitement des interruptions (IO + capteur de hall)
 - Vérification de la tension d'alimentation
 - Réception d'un « reset » depuis le circuit d'affichage
 - Traitement de l'auto-play (écouter une plage)
 - Traitement « stop à zéro » et mise à jour de l'affichage

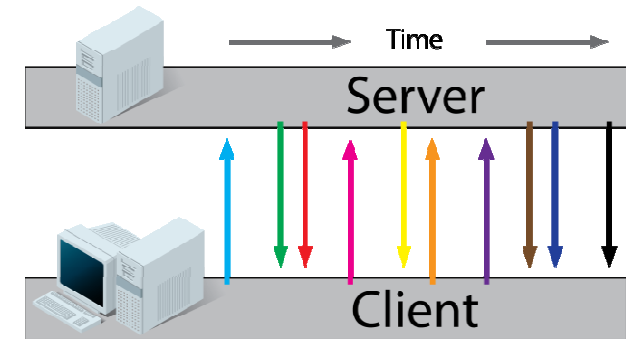
Télécommande Revox B77

Librairies

- Librairies utilisées (on ne re-invente pas tout)
 - Webserver, websockets (librairie ESP existante)
 - Update par wifi en ligne OTA (librairie ESP)
 - Interface MCP (librairie Adafruit)
 - Librairie Arduino-JSON
 - Librairie Bootstrap (dans data)



Websocket Utilisation



- Fonctionnement avec JSON

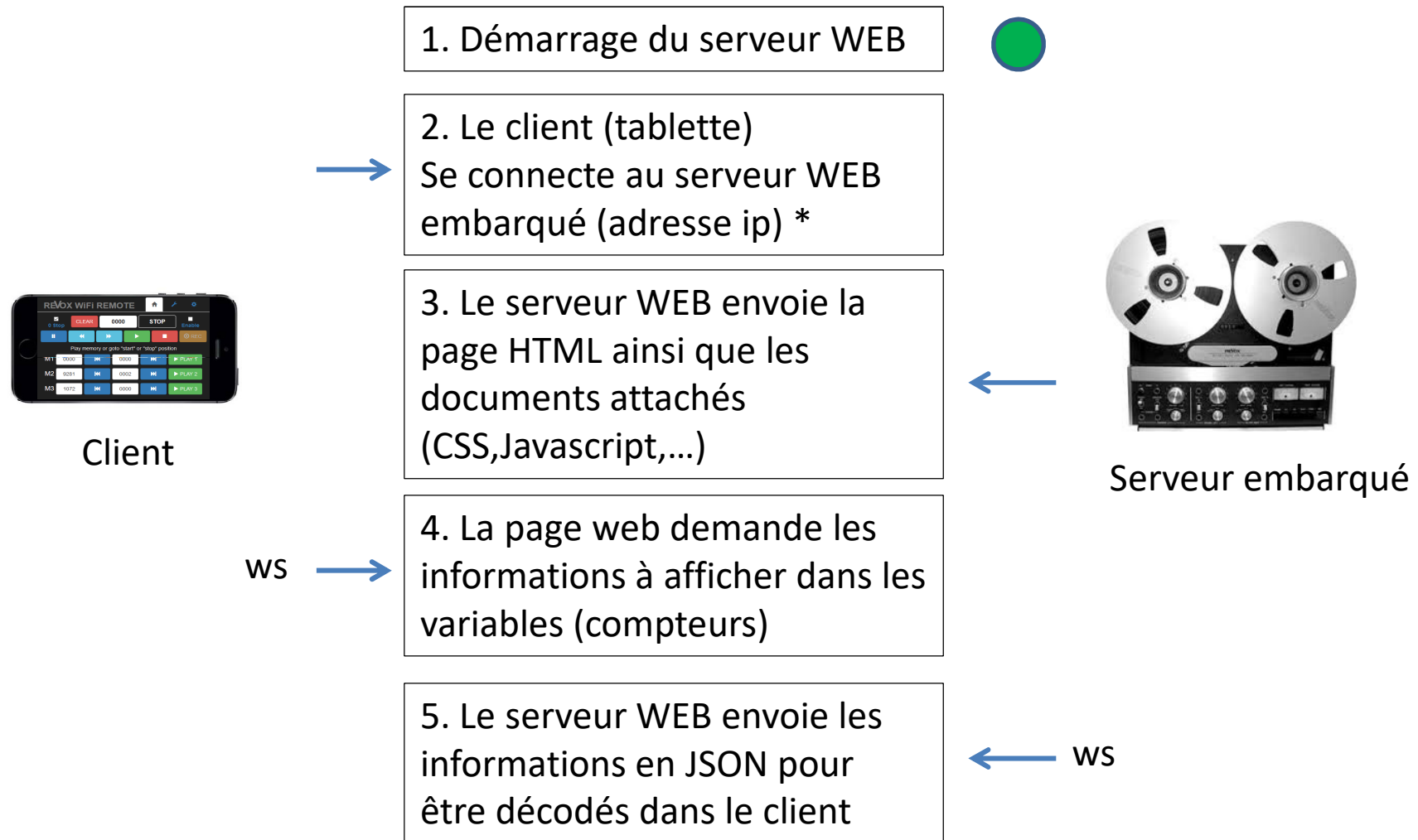
Exemple:

```
{"id1": "valeur1", "id2": "valeur2", "idn": "valeurn"};
```

- On peut donc envoyer des messages de longueur variable
- Utilisation:
 - Message court pour mise à jour du compteur
 - Message long pour mise à jour des positions

Communication

Web-Server → HTML



Démarrage du code HTML dans le client

- Code exécuté dans le browser

```
<html>
<head> <title>Revox Wifi Remote</title>
.... Code d'initialisation de la page sur le client
<link rel="stylesheet" href="css/bootstrap.min.css">
<script src="js/jquery.min.js"></script>
<script src="js/bootstrap.min.js"></script>
<script src="WebSocket.js" type="text/javascript"></script>
<style>
..... code CSS pour le formatage dépendant de l'appareil (smartphone, tablette, pc,..)
</style>
</head>
<boddy>
.....
<button type="button" id="$G3" class="btn btn-primary" onclick="sendButton(this.id)">
...

</body>
</html>
```

ms.onopen est exécuté

Websocket (1)

mise à jour au démarrage

- Code exécuté dans le browser

```
ws.onopen = function(evt){  
    ws.send('Connect ' + new Date());  
    ...  
    updateCounters();  
};
```

! demande des données du serveur

```
function updateCounters(){  
    $.getJSON('/update.json', function(data){  
  
        $('#mS1').html(pad(data.mS1,4));  
        ....  
        $('#nE3').val(pad(data.mE3,4));  
  
    }).fail(function(err){  
        console.log("err getJSON update.json "+JSON.stringify(err));  
    });  
};
```

La fonction pad ajoute des 0000 devant le nombre à afficher

! tag HTML

! Donnée reçue du serveur

WebSocket

Initialisation du serveur

- Code C dans ESP8266:

```
void startServer() { // Démarrage du serveur WEB Embarqué (dans setup()).
```

```
.....
```

identifiant

Fonction à exécuté

```
server.on("/update.json", updateCounters);
```

```
server.on("/offset.json", updateOffsets);
```

```
server.onNotFound(handleNotFound);
```

```
server.begin();    // start the HTTP server
```

```
Serial.println("HTTP server started.");
```

```
}
```

Envoi des compteurs depuis le serveur WEB

- Code C sur ESP8266 (réponse à une demande de la page HTML)

```
updateCounters() {  
    // préparation du texte à renvoyer en format JSON  
    String json = "{\"mS1\": \"\" + String(cntS[0]) + "\", \"mE1\": \"\" + String(cntE[0]) + "\", \"mS2\": \"\" + String(cntS[1]) + "\", \"mE2\": \"\" + String(cntE[1]) + "\", \"mS3\": \"\" + String(cntS[2]) + "\", \"mE3\": \"\" + String(cntE[2]) + \"\"}";  
    // envoi vers le client WEB  
    server.send(200, "application/json", json);  
    json="";  
}
```

Websocket (2)

Action sur pression d'un bouton

- Code exécuté sur le browser (client)

<html>

.....

.....

</head>

<body>

.....

<button type="button" id="\$CL" class="btn btn-danger" onclick="sendButton(this.id)" >CLEAR</button>

.....

</body>

</html>

Identifiant du bouton

Routine JS à appeler

Formatage « bootstrap »

Envoyer un élément (bouton) vers le serveur web

- Code JS exécuté dans le browser

```
function sendButton(obj){  
  if(obj=="$CL" ){  
    if(!confirm("Are you sure!")){  
      return;  
    }  
  }  
  ws.send(obj);  
  console.log(obj);  
};
```

→ Fonction utilisée pour tous les boutons

→ S'il s'agit de la fonction « CLEAR » nous demandons confirmation

→ L'information est envoyée au serveur

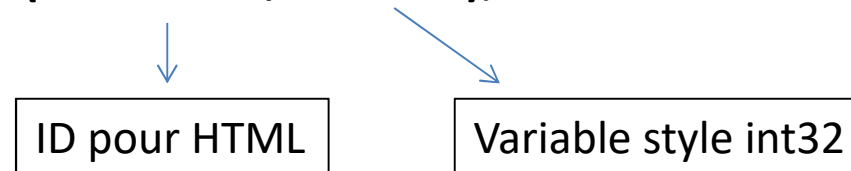
Websockets (3)

Mise à jour du compteur

- Code C exécuté sur l'ESP8266

```
void updateCounter( String(id),int (data)) {  
  
String json = "{\"id\": \"" + id + "\",\"  
              + \"count\": \"" + String(data) + "\"}";  
  
WebSocket.broadcastTXT(json);  
}
```

Appel de la fonction dans notre code principal:
updateCounter("maincnt",mainCnt);



Récupération dans la page WEB

- Code exécuté dans le browser

```
<span id="mainCnt" class="label label-default">COUNTER</span>
```

Formatage « bootstrap »

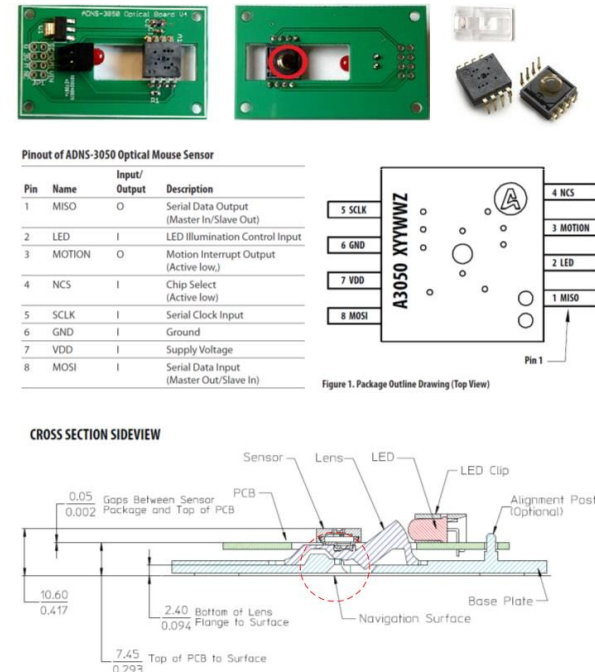
- Dans javascript (chargé avec html)

```
ws.onmessage = function (evt) {  
    var data = JSON.parse(evt.data);  
    if((data.id)=="mainCnt"){ // envoyer le chiffre sur le compteur  
        document.getElementById(data.id).innerHTML = pad(data.count,4);  
    } else if((data.id) == ...  
}  
};
```

- La fonction pad ajoute des 0 (zéro) devant le chiffre pour

Prochaine étape

- Développer un compteur temps réel avec un capteur optique de souris
- ADNS3050 gaming sensor
- Caractéristiques
 - 60 IPS (inch per seconds)
 - 2000 cpi
 - Accel. 20g
- Mesure sans contacts !



Démonstration

